

HW9

Group 12 - Shir, Omer & Ido

16/06/26

Group:

Group members (Name, ID, email)

- Ido Ravid, 207198177, ido.ravid@mail.huji.ac.il
 - Omer Sutovsky, 318735107, omer.sutovsky@mail.huji.ac.il
 - Shir Tsarfaty, 207718008, shir.tsarfaty@mail.huji.ac.il
-

Paths and Data

```
if (!require('data.table')) { install.packages('data.table'); library('data.table') }
if (!require('splines'))      { install.packages('splines');   library('splines') }
if (!require('MASS'))         { install.packages('MASS');      library('MASS') }
```

```
source("../utils.R")
source("../lab5/lab5_utils.R")
```

```
healthy_file <- "../lab2/data/TCGA-13-0723-10B_lib2_all_chr1.forward"
tumor_file   <- "../lab7/data/TCGA-13-0723-01A_lib2_all_chr1.forward"
chr1_file    <- "../lab1/data/chr1_str.rda"
```

```
healthy_reads <- fread(healthy_file, col.names = c("Chr", "Loc", "Length"))
tumor_reads   <- fread(tumor_file,   col.names = c("Chr", "Loc", "Length"))
load(chr1_file)
```

```
BIN <- 2500
```

```
add_pos <- function(df, beg) {
  df$pos <- beg + (seq_len(nrow(df)) - 1) * BIN + BIN / 2
  df
}
```

```
# Two arms around centromere (120-130 Mb excluded), both samples
```

```
df_h <- rbind(
  add_pos(bin_data(healthy_reads, chr1, 5000001L, 119990000L, BIN), 5000001L),
  add_pos(bin_data(healthy_reads, chr1, 130000001L, 242000000L, BIN), 130000001L)
)
df_t <- rbind(
  add_pos(bin_data(tumor_reads, chr1, 5000001L, 119990000L, BIN), 5000001L),
  add_pos(bin_data(tumor_reads, chr1, 130000001L, 242000000L, BIN), 130000001L)
)
```

```

# Per-sample GC spline + plug-in estimator (lab 8 method b)
eps <- 1
gc_correct <- function(df) {
  nz <- df$reads > 0
  fit <- fit_gc_spline(df[nz, ])
  fhat <- pmax(predict(fit, data.frame(gc = df$gc)), 0)
  raw <- (df$reads + eps) / (fhat + eps)
  raw / median(raw[nz])
}
df_h$cn <- gc_correct(df_h)

## Warning in bs(gc, degree = 3L, knots = c(`25%` = 0.3684, `50%` = 0.4064, : some
## 'x' values beyond boundary knots may cause ill-conditioned bases

df_t$cn <- gc_correct(df_t)

## Warning in bs(gc, degree = 3L, knots = c(`25%` = 0.3684, `50%` = 0.4064, : some
## 'x' values beyond boundary knots may cause ill-conditioned bases

# Two-sample estimator (lab 8 method d): GC-corrected tumor / GC-corrected healthy
df_t$cn_2s <- df_t$cn / df_h$cn

nz_h <- df_h$reads > 0
nz_t <- df_t$reads > 0
nz <- nz_h & nz_t # bins with reads in both samples

# alias so rest of code uses df / df$cn referring to working data frame
df <- df_t

cat(sprintf("Total bins: %d | both non-zero: %d\n", nrow(df), sum(nz)))

## Total bins: 90796 | both non-zero: 85010

```

Part A – Test Statistic

We need a statistic T that summarises whether a genomic region has more or fewer reads than expected under copy number 1, while being **resistant to single noisy bins** – meaning one extreme bin should not be able to drive the entire region to significance on its own.

We use the **two-sample GC-corrected estimator** (Lab 8 method d): $\hat{a}_k = \hat{a}_k^{\text{tumor}} / \hat{a}_k^{\text{healthy}}$, where each sample is corrected by its own GC spline and median-normalised. Dividing cancels shared GC bias and library-specific technical effects, leaving a cleaner copy-number signal than single-sample correction alone.

We then take $\log_2(\hat{a}_k)$: under H_0 (CN = 1) this is centred at 0, gains are positive, losses are negative, and the $\log_2$ scale has a natural interpretation (a value of 1 means a doubling; -1 means a halving).

We considered three candidates, all computed as a rolling aggregate over a window of $W = 11$ bins (≈ 27.5 kb):

Statistic	Formula	Notes
S1: plain $\log_2$	$\log_2(\hat{a}_k)$ – per bin, no aggregation	Baseline
S2: trimmed mean	$\log_2(\hat{a})_{10\%}$ over window	Hard-discards top/bottom 10% of bins

Statistic	Formula	Notes
S3 : Huber location	$\hat{\mu}_{\text{Huber}}(\log_2(\hat{a}), k = 1.345)$ over window	Continuously downweights outlier bins

We evaluated all three against the empirical null (Part B). S2 and S3 produce noticeably narrower, more compressed null distributions when combined with the local-stability negative control filter: the filter already removes noisy bins, so a further robust aggregation over-compresses the tails. S1 – plain $\log_2(\hat{a}_k)$ – keeps the most natural spread. We carry all three forward to Part C and defer the final choice until we see their detection performance on the full chromosome; the “resistance to single noisy bins” requirement is in any case addressed by the negative control selection (Part B) rather than by the statistic alone.

```

W      <- 11
HALF   <- (W - 1) / 2

log2_cn <- log2(pmax(df$cn_2s, 1e-6))  # log2 of two-sample a-hat

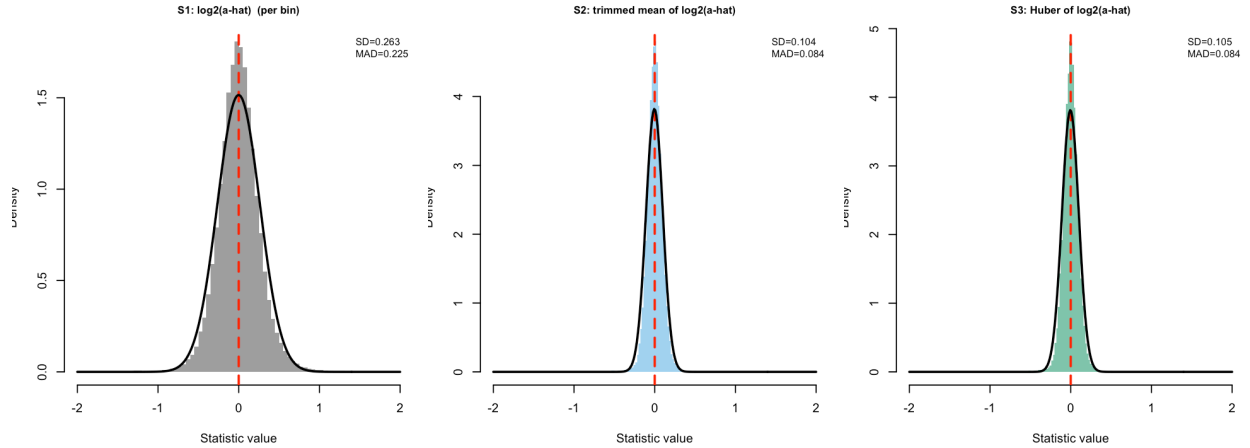
# Rolling window helper (kept for S2/S3 comparison plots)
roll_stat <- function(x, fun) {
  out <- rep(NA_real_, length(x))
  for (i in seq_along(x)) {
    idx <- max(1, i - HALF) : min(length(x), i + HALF)
    sub <- x[idx][nz[idx]]
    if (length(sub) >= 5) out[i] <- fun(sub)
  }
  out
}

df$S1 <- log2_cn
df$S2 <- roll_stat(log2_cn, function(x) mean(x, trim = 0.10))
df$S3 <- roll_stat(log2_cn, function(x) huber(x, k = 1.345)$mu)

par(mfrow = c(1, 3), mar = c(4, 3.5, 2.5, 1))
stats <- list(
  list(v = df$S1[nz], lab = "S1: log2(a-hat) (per bin)",
    list(v = df$S2[nz & !is.na(df$S2)], lab = "S2: trimmed mean of log2(a-hat)"),
    list(v = df$S3[nz & !is.na(df$S3)], lab = "S3: Huber of log2(a-hat)")
)
cols <- c("#555555", "#56B4E9", "#009E73")

for (i in seq_along(stats)) {
  v      <- stats[[i]]$v
  v      <- pmin(pmax(v, -2), 2)
  xlim   <- c(-2, 2)
  xseq   <- seq(xlim[1], xlim[2], length.out = 300)
  h      <- hist(v, breaks = 100, plot = FALSE)
  plot(h, freq = FALSE, col = adjustcolor(cols[i], 0.6), border = NA,
    main = stats[[i]]$lab, xlab = "Statistic value", cex.main = 0.85,
    xlim = xlim)
  abline(v = 0, col = "red", lwd = 2, lty = 2)
  lines(xseq, dnorm(xseq, mean(v, na.rm = TRUE), sd(v, na.rm = TRUE)),
    col = "black", lwd = 2)
  legend("topright", bty = "n", cex = 0.8,
    legend = sprintf("SD=%.3f\nMAD=%.3f", sd(v, na.rm=TRUE), mad(v, na.rm=TRUE)))
}

```



Part B – Negative Control Regions and Null Distribution

How we select the negative controls. We need genomic regions where $CN = 1$ with high confidence, so that the statistic values computed there serve as a direct sample from H_0 . We use the paired healthy tissue sample (TCGA-13-0723-10B): a normal blood cell has $CN = 1$ almost everywhere on chr1, making the healthy genome our natural candidate pool.

We apply the local stability filter (**Strategy B**) directly on the **two-sample ratio** $\log_2(\hat{a}_k)$: a bin is kept as a negative control only if its 11-bin neighbourhood simultaneously has a median near 0 **and** a low local SD. We set the two cut-offs in interpretable linear-ratio units and then convert to $\log_2$: the local median CN must stay within $\pm 10\%$ of the no-change baseline ($|\tilde{T}| \leq \log_2 1.10 \approx 0.137$), and the local scatter must stay under $\pm 15\%$ ($SD \leq \log_2 1.15 \approx 0.202$). Filtering on the ratio rather than the healthy sample alone is the correct choice: H_0 is “no tumor-specific CN change” (ratio = 1), not “healthy CN = 1 absolutely.” This also handles germline CNVs correctly — if both samples carry a duplication at some locus, the ratio is still 1 and the bin is a valid negative control. Bins at tumor CN-event boundaries are excluded because their neighbours reveal the disruption even if the bin itself lands near 1. Approximately 37% of non-zero bins pass this filter, giving $\sim 31,000$ control bins across both chromosome arms.

How we model H_0 . The $\log_2(\hat{a}_k)$ values of the selected control bins are a direct sample from the distribution of the statistic when $CN = 1$ – we use them directly as the empirical null, without assuming any parametric form. The histograms and QQ plots below characterise this distribution for each of the three candidate statistics.

How p-values are derived (Part C). For any test bin k with statistic value T_k , the two-sided empirical p-value is the fraction of null values at least as extreme:

$$p_k = \frac{|\{j \in \text{ctrl} : |T_j| \geq |T_k|\}|}{|\text{ctrl}|}$$

With $\sim 31,000$ control bins this resolves p-values down to $\approx 3 \times 10^{-5}$, sufficient for testing at $\alpha = 0.01$ after multiple testing correction. No normality assumption is required.

```
# Stability filter runs on the two-sample ratio (log2_cn = log2(tumor/healthy)).
# H0 is "no tumor-specific CN change" (ratio = 1), so the filter should assess
# whether the ratio is locally stable near 0 -- not whether the healthy alone is.
# This also correctly handles germline CNVs: if both samples have CN=2, ratio=1
# and the bin is still a valid negative control.
# Thresholds set in interpretable linear-ratio units, then converted to log2:
```

```

# median within +/-10% of the no-change baseline; local scatter under +/-15%.
med_thresh <- log2(1.10) # ~0.137
sd_thresh  <- log2(1.15) # ~0.202

win_med_log <- roll_stat(log2_cn, median)
win_sd_log  <- roll_stat(log2_cn, sd)
stratB <- nz & !is.na(win_med_log) & !is.na(win_sd_log) &
  abs(win_med_log) <= med_thresh & win_sd_log <= sd_thresh

cat(sprintf("Strategy B (local stability): %d bins (%.0f%% of non-zero)\n",
  sum(stratB), 100 * sum(stratB) / sum(nz)))

```

```
## Strategy B (local stability): 30959 bins (36% of non-zero)
```

```

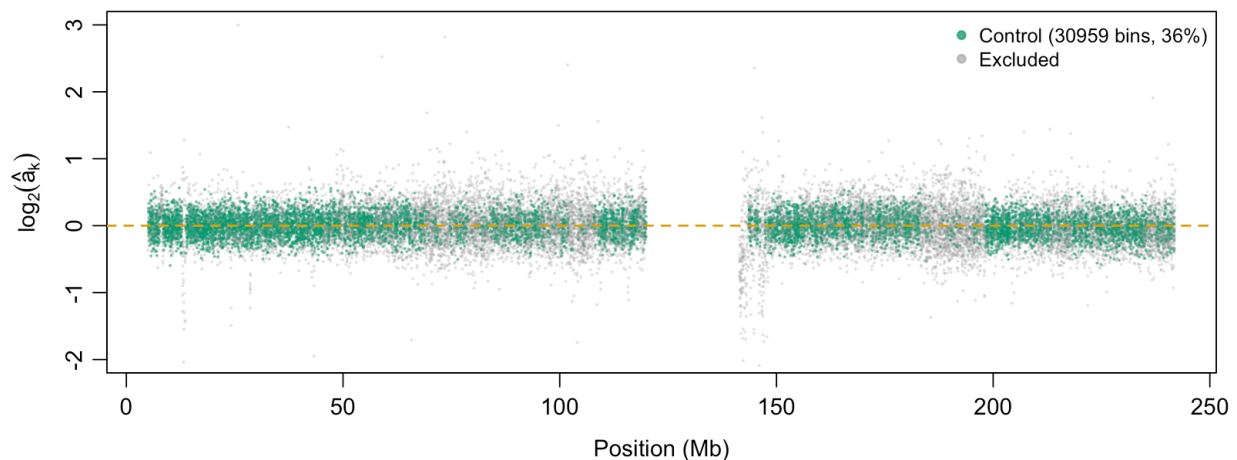
# Chromosome-wide scatter: control bins highlighted, excluded in grey
set.seed(42)
idx_all <- which(nz)
s <- sample(idx_all, min(25000, length(idx_all)))
x_mb <- df$pos[s] / 1e6
y_cn <- pmin(log2_cn[s], 3)

col_vec <- ifelse(stratB[s],
  adjustcolor("#009E73", 0.5),
  adjustcolor("#999999", 0.2))

par(mar = c(4.5, 4, 2.5, 1), mgp = c(2.2, 0.7, 0))
plot(x_mb, y_cn, pch = 20, cex = 0.25, col = col_vec,
  xlab = "Position (Mb)", ylab = expression(log[2](hat(a)[k])),
  main = "Negative control bins across chr1 (green = control, grey = excluded)",
  ylim = c(-2, 3))
abline(h = 0, col = "#E69F00", lwd = 1.5, lty = 2)
legend("topright", bty = "n", cex = 0.85,
  legend = c(sprintf("Control (%d bins, %.0f%%)",
    sum(stratB), 100 * sum(stratB) / sum(nz)), "Excluded"),
  col = c(adjustcolor("#009E73", 0.8), adjustcolor("#999999", 0.6)),
  pch = 20, pt.cex = 1.2)

```

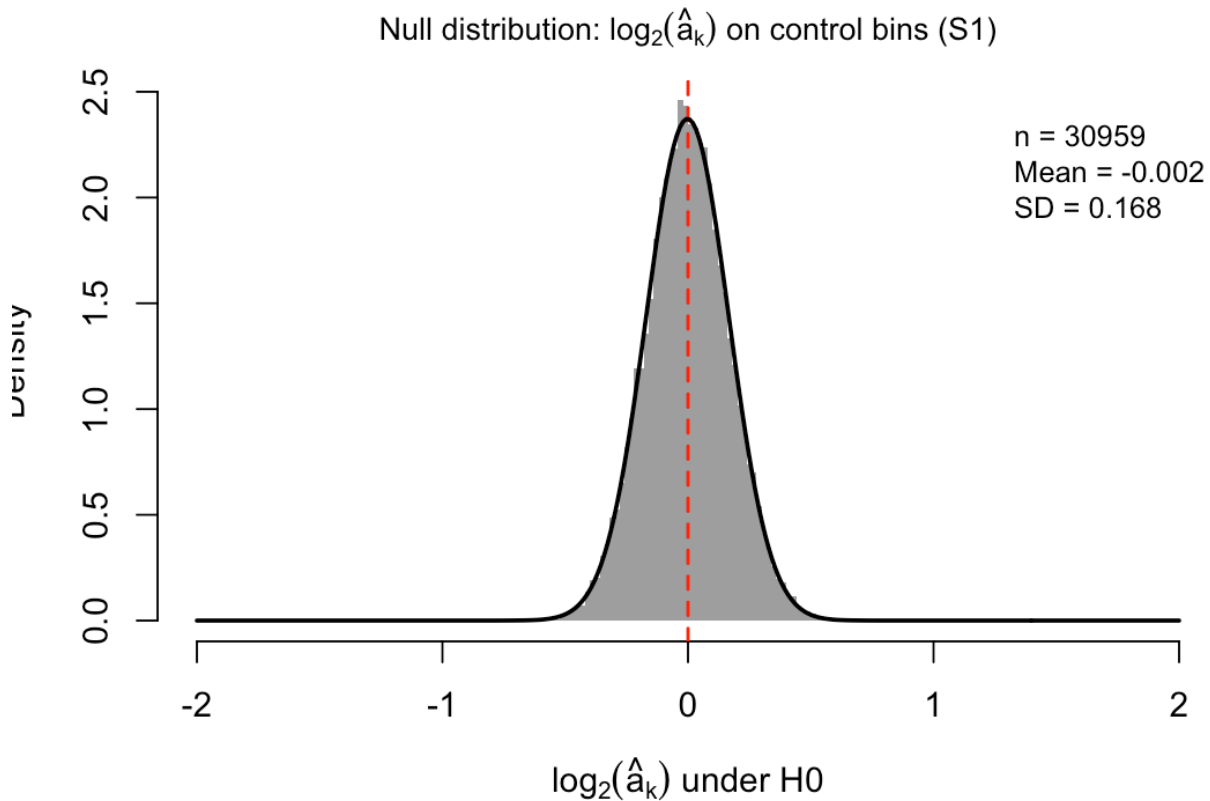
Negative control bins across chr1 (green = control, grey = excluded)



```

# Null distribution for S1 on control bins, with normal overlay
null_s1 <- df$S1[stratB & !is.na(df$S1)]
xlim    <- c(-2, 2)
xseq    <- seq(xlim[1], xlim[2], length.out = 300)
h       <- hist(null_s1, breaks = 80, plot = FALSE)
par(mar = c(4, 3.5, 2.5, 1))
plot(h, freq = FALSE, col = adjustcolor("#555555", 0.6), border = NA,
     main = expression("Null distribution: " * log[2](hat(a)[k]) * " on control bins
      ↪ (S1)"),
     xlab = expression(log[2](hat(a)[k]) * " under H0"),
     xlim = xlim, cex.main = 0.9)
abline(v = 0, col = "red", lwd = 1.5, lty = 2)
lines(xseq, dnorm(xseq, mean(null_s1), sd(null_s1)), col = "black", lwd = 2)
legend("topright", bty = "n", cex = 0.85,
      legend = sprintf("n = %d\nMean = %.3f\nSD = %.3f",
        length(null_s1), mean(null_s1), sd(null_s1)))

```



```

stat_list_b <- list(S1 = df$S1, S2 = df$S2, S3 = df$S3)
stat_labs_b <- c("S1: log2(a-hat)", "S2: trimmed mean", "S3: Huber")
cols_b     <- c("#555555", "#56B4E9", "#009E73")

par(mfrow = c(2, 3), mar = c(4.5, 4, 3, 1), mgp = c(2.4, 0.8, 0),
    cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)

for (ti in 1:3) {
  mask <- stratB & !is.na(stat_list_b[[ti]])
  v    <- stat_list_b[[ti]][mask]

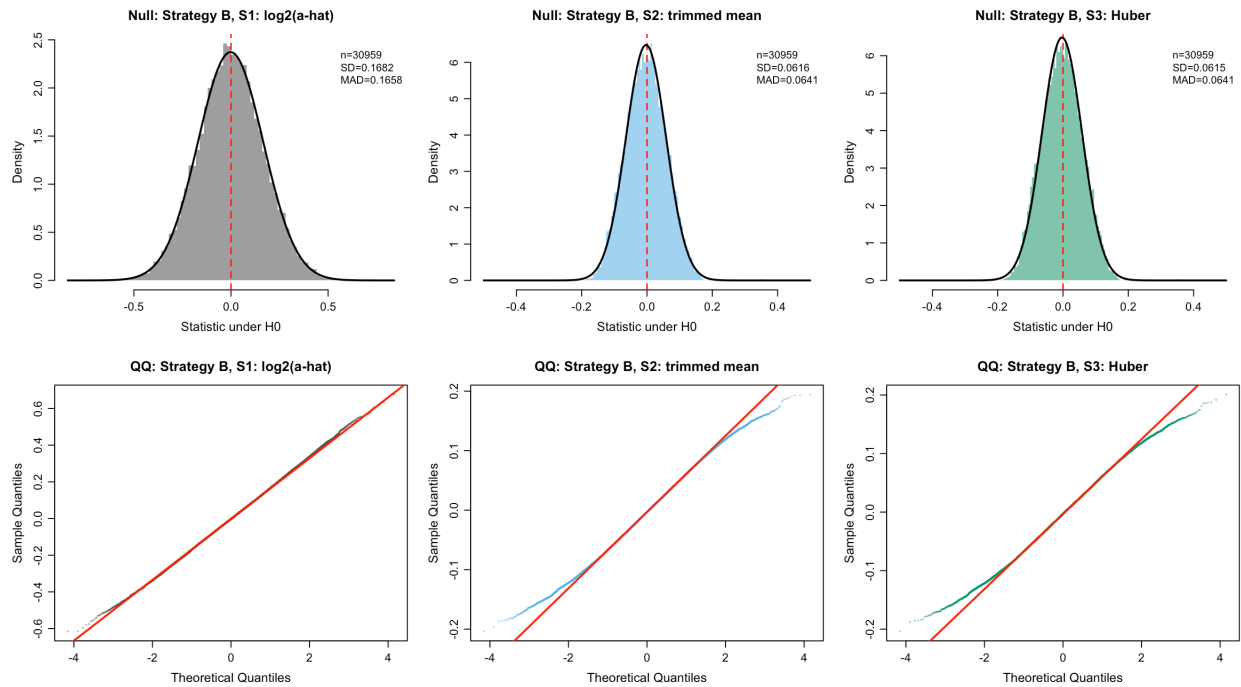
```

```

# xlim: 5 SDs each side so shape is always visible, floor at ±0.5
hw    <- max(5 * sd(v), 0.5)
xlim  <- c(-hw, hw)
xseq  <- seq(xlim[1], xlim[2], length.out = 300)
h     <- hist(v, breaks = 80, plot = FALSE)
plot(h, freq = FALSE, col = adjustcolor(cols_b[ti], 0.6), border = NA,
     main = sprintf("Null: Strategy B, %s", stat_labs_b[ti]),
     xlab = "Statistic under H0", xlim = xlim)
abline(v = 0, col = "red", lwd = 1.5, lty = 2)
lines(xseq, dnorm(xseq, mean(v), sd(v)), col = "black", lwd = 2)
legend("topright", bty = "n", cex = 1.0,
      legend = sprintf("n=%d\nSD=%.4f\nMAD=%.4f", length(v), sd(v), mad(v)))
}

for (ti in 1:3) {
  mask <- stratB & !is.na(stat_list_b[[ti]])
  v    <- stat_list_b[[ti]][mask]
  qqnorm(v, pch = 20, cex = 0.3, col = adjustcolor(cols_b[ti], 0.4),
        main = sprintf("QQ: Strategy B, %s", stat_labs_b[ti]))
  qqline(v, col = "red", lwd = 2)
}

```



The histograms are shown with a fitted normal overlay (black line) for reference, and the QQ plots assess normality directly. S2 and S3 are visibly narrower and more compressed than S1, a consequence of applying robust aggregation on top of bins already filtered for stability. But comparing shapes to a normal curve is only superficial – what actually matters is whether each statistic produces **calibrated** p-values. We test this directly below.

Validating calibration (split-half)

A statistic is only usable if, on data that truly follows H_0 , its p-values are **Uniform(0,1)** – equivalently, the realised false-positive rate at level α equals α . We can check this with the control bins alone, no labelled

events required:

1. Split the control bins into **interleaved genomic blocks** (block size 100 bins $\gg$ window $W = 11$, so the two sets never share a window). Even blocks build the empirical null; odd blocks are held out as “test” bins.
2. The held-out bins are themselves $CN = 1$, so under a correctly calibrated test their two-sided empirical p-values must be uniform.
3. We measure the deviation from uniform (Kolmogorov–Smirnov D) and the realised false-positive rate at $\alpha = 0.01$ and 0.05 .

A statistic whose null is artificially narrow (S2/S3) will make ordinary $CN = 1$ fluctuations look significant, inflating the false-positive rate above α – which disqualifies it regardless of how its histogram looks.

```
# Interleaved-block split so null-building and test bins never share a window
ctrl_idx <- which(stratB)
blk      <- 100                                # block size (bins) >> W = 11
block_id <- ctrl_idx %/% blk
null_ix  <- ctrl_idx[block_id %% 2 == 0] # even blocks -> build null
test_ix  <- ctrl_idx[block_id %% 2 == 1] # odd blocks -> held-out CN=1 tests

# Add-one two-sided empirical p-value (never zero, valid by construction)
emp_p2 <- function(t_test, t_null) {
  n <- length(t_null)
  at <- abs(t_test)
  vapply(at, function(a) (1 + sum(abs(t_null) >= a)) / (n + 1), numeric(1))
}

p_lists <- list()
calib_rows <- list()
for (ti in seq_along(stat_list_b)) {
  v_null <- stat_list_b[[ti]][null_ix]; v_null <- v_null[is.finite(v_null)]
  v_test <- stat_list_b[[ti]][test_ix]; v_test <- v_test[is.finite(v_test)]
  m <- median(v_null)                                # H0 centre, estimated from the null
  p <- emp_p2(v_test - m, v_null - m)
  p_lists[[ti]] <- p
  calib_rows[[ti]] <- data.frame(
    Statistic = names(stat_list_b)[ti],
    n_test    = length(p),
    FPR_0.01  = round(mean(p < 0.01), 4),
    FPR_0.05  = round(mean(p < 0.05), 4),
    KS_D      = round(suppressWarnings(ks.test(p, "punif")$statistic), 4)
  )
}
calib_tbl <- do.call(rbind, calib_rows)
rownames(calib_tbl) <- NULL
knitr::kable(calib_tbl,
  col.names = c("Statistic", "n (test)", "FPR @ 0.01", "FPR @ 0.05", "KS D"),
  caption = "Calibration on held-out CN=1 bins. Well-calibrated: FPR approx
  ↪ alpha, KS D small.")
```

Table 1: Calibration on held-out $CN=1$ bins. Well-calibrated: FPR approx α , KS D small.

Statistic	n (test)	FPR @ 0.01	FPR @ 0.05	KS D
S1	15633	0.0086	0.0496	0.0088

Statistic	n (test)	FPR @ 0.01	FPR @ 0.05	KS D
S2	15633	0.0077	0.0430	0.0155
S3	15633	0.0090	0.0440	0.0137

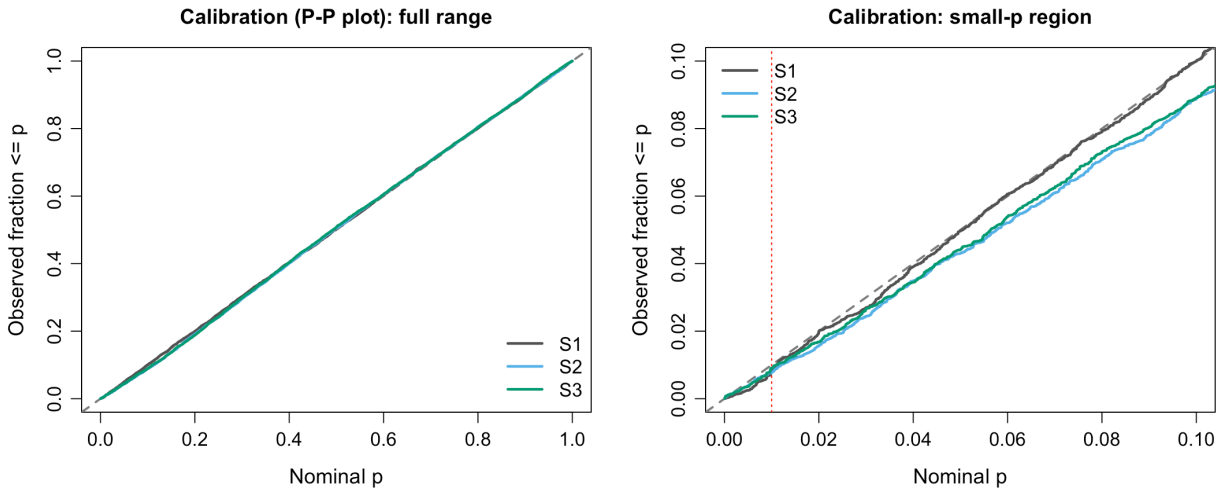
```

cols_b <- c("#555555", "#56B4E9", "#009E73")
par(mfrow = c(1, 2), mar = c(4.5, 4.5, 3, 1), mgp = c(2.6, 0.8, 0),
    cex.main = 1.2, cex.lab = 1.2, cex.axis = 1.1)

# Full P-P plot: empirical CDF of p-values vs uniform diagonal
plot(NA, xlim = c(0, 1), ylim = c(0, 1),
     xlab = "Nominal p", ylab = "Observed fraction <= p",
     main = "Calibration (P-P plot): full range")
abline(0, 1, col = "grey50", lwd = 2, lty = 2)
for (ti in 1:3) {
  ps <- sort(p_lists[[ti]])
  lines(ps, (seq_along(ps)) / length(ps), col = cols_b[ti], lwd = 2.5)
}
legend("bottomright", bty = "n", cex = 1.1, lwd = 2.5, col = cols_b,
      legend = names(stat_list_b))

# Zoom into the testing-relevant small-p region
plot(NA, xlim = c(0, 0.1), ylim = c(0, 0.1),
     xlab = "Nominal p", ylab = "Observed fraction <= p",
     main = "Calibration: small-p region")
abline(0, 1, col = "grey50", lwd = 2, lty = 2)
abline(v = 0.01, col = "red", lwd = 1.2, lty = 3)
for (ti in 1:3) {
  ps <- sort(p_lists[[ti]])
  lines(ps, (seq_along(ps)) / length(ps), col = cols_b[ti], lwd = 2.5)
}
legend("topleft", bty = "n", cex = 1.1, lwd = 2.5, col = cols_b,
      legend = names(stat_list_b))

```



Result. All three statistics are calibrated: every realised false-positive rate sits at or below its nominal α , and the P-P curves track the diagonal. This is expected – empirical p-values are self-calibrating by construction, because each statistic is ranked against *its own* null, so the over-compression of S2/S3 cancels

between the null and the test bins. The narrow histograms of S2/S3 therefore do **not** cause false positives; superficial non-normality was a red herring.

The check still discriminates, though. **S1 is the best calibrated** (KS $D \approx 0.009$, with FPR almost exactly 0.01 and 0.05), tracking the uniform diagonal nearly perfectly. S2 and S3 are slightly **conservative** – their realised FPR falls *below* α and their P–P curves bow under the diagonal – because the heavy aggregation discretises the statistic and inflates ties. Conservatism is safe but costs power: at a fixed α , a conservative test detects fewer true events. Calibration alone thus does not eliminate any candidate; it ranks S1 marginally best and sets up the decisive test – **detection power** – which we examine next.

External sources statement

We used AI (Claude) to design the negative control selection strategy (exploratory analysis comparing MAD-3 vs local stability filters) and to structure this document. Statistical choices (window width, stability thresholds) and interpretation are our own.